Optimal Path Planning with Soft Homology Constraints*

Shammas Ahmed Sebastian Perez-Salazar

Working draft, last modified October 23, 2026

Abstract

We study shortest-path planning under soft homology constraints on a surface that is topologically a disk with holes. We model the problem as a flow problem and give a linear program whose optimal integral solutions are shortest (s,t)-walks in the homology class of a reference path. In practice these solutions may contain detached cycles, so we formalize the Homologous-Simple-Path (HSP) problem, which asks for a shortest homologous *simple* path, and give a reduction sketch indicating that it is NP-hard. We propose a polynomial-time heuristic, ALG, show that it preserves the homology class, and show that it admits no bounded approximation ratio. We also describe two exact, exponential-time formulations based on MTZ constraints and cycle cuts. The exact complexity of HSP and the existence of heuristics with provable guarantees remain open.

1 Introduction

Taveras et al. [1] treat homology constraints on paths via a Lagrangian relaxation that penalizes the difference in homology between a reference path and the proposed path. This is simple to implement, but it relies on a penalty parameter that must be tuned on an instance-by-instance basis; for insufficiently large penalties the resulting solution is infeasible, i.e., it is not in the same homology class as the reference path.

In this draft we instead impose the homology constraint exactly and study the resulting optimization problem. Our contributions, all of which are partial, are as follows.

- A flow-based integer program, and an LP relaxation, for the shortest path homologous to a reference path (Section 2).
- An observation that the LP can return walks with detached cycles, and a formalization of the simple-path variant HSP, together with a reduction sketch suggesting it is NP-hard (Section 2.3).
- A polynomial-time heuristic ALG that preserves the homology class, together with a proof that it has no bounded approximation ratio (Sections 3 and 4).
- Two exact (exponential-time) formulations of HSP (Section 5).

*This is an unfinished working draft. It is posted to document the approach, the negative results obtained, and the questions that remain open.

We close with a list of open problems (Section 6).

2 Problem Statement

Let $\mathcal{X} = (\mathcal{V}, \mathcal{E}, \mathcal{T})$ be a 2-dimensional disk-like simplicial complex with D holes, with $|\mathcal{E}| = m$ oriented edges and $|\mathcal{T}| = k$ triangles. Let $s, t \in \mathcal{V}$ be the source and destination, and let $\bar{\tau} = (s, v_1, \ldots, v_n, t)$ be a reference path. Let ∂_2 be the boundary matrix mapping triangles to edges, and let H be the matrix whose columns form a basis of the harmonic 1-chains of $\mathcal{X}$, so that the entries of $H^\top \tau$ record how a chain τ winds around each of the D holes. We wish to compute a minimum-length path τ^* homologous to $\bar{\tau}$, i.e., to solve

$$\begin{aligned} \min_{\tau} \quad & c(\tau) \\ \text{s.t.} \quad & \tau = \bar{\tau} + \partial_2 \mu \\ & \tau \in \mathbb{Z}^m, \ \mu \in \mathbb{Z}^k, \end{aligned} \tag{1}$$

where $c(\tau)$ is the cost (length) of the (s,t)-dipath τ.

2.1 As a Flow Problem

Equivalently, we can formulate the problem as a unit integer flow with a hard constraint that the flow lie in the homology class of the reference flow. Writing $\delta^-(v)$ and $\delta^+(v)$ for the edges entering and leaving v, we obtain

$$\begin{aligned} \min \quad & \sum_{e \in \mathcal{E}} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta^-(v)} x_e - \sum_{e \in \delta^+(v)} x_e = \begin{cases} -1 & v = s, \\ 1 & v = t, \\ 0 & \text{otherwise} \end{cases} && \text{(flow conservation)} \\ & H^\top \tau(x) = H^\top \bar{\tau} && \text{(homology constraint)} \\ & x_e \in \{0, 1\} \quad (\forall e \in \mathcal{E}). \end{aligned} \tag{2}$$

Lemma 1. *Problems (1) and (2) are equivalent formulations of the optimal homologous path problem.*

Proof. The objectives coincide, so we show that the feasible regions coincide. Suppose a path τ is feasible for (1) for some reference path $\bar{\tau}$. By the argument in Section II.C of [1], $H^\top \tau = H^\top \bar{\tau}$. Flow conservation and the unit constraints hold because τ is an (s,t)-dipath. Hence τ is feasible for (2). Conversely, suppose τ is feasible for (2). It is an (s,t)-dipath that is homologous to $\bar{\tau}$, so by the definition of homology it can be written as $\tau = \bar{\tau} + \partial_2 \mu$ for some integral μ. Hence τ is feasible for (1). □

2.2 The Relaxed Problems

We relax (1) as

$$\begin{aligned} \min_{\tau} \quad & c(\tau) \\ \text{s.t.} \quad & \tau = \bar{\tau} + \partial_2 \mu \\ & \tau \in \mathbb{R}^m, \ \mu \in \mathbb{R}^k. \end{aligned} \tag{1_{LP}}$$

By Theorem 4.1 of [2], ∂_2 is totally unimodular. It follows that the constraint matrix $[\, I \quad -\partial_2 \,]$ is totally unimodular, so (1_{LP}) has an integral optimal solution, which is optimal for (1).

We relax (2) as

$$\begin{aligned} \min \quad & \sum_{e \in \mathcal{E}} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta^-(v)} x_e - \sum_{e \in \delta^+(v)} x_e = \begin{cases} -1 & v = s, \\ 1 & v = t, \\ 0 & \text{otherwise} \end{cases} \\ & H^\top \tau(x) = H^\top \bar{\tau} \\ & x_e \in [0,1] \quad (\forall e \in \mathcal{E}). \end{aligned} \tag{2_{LP}}$$

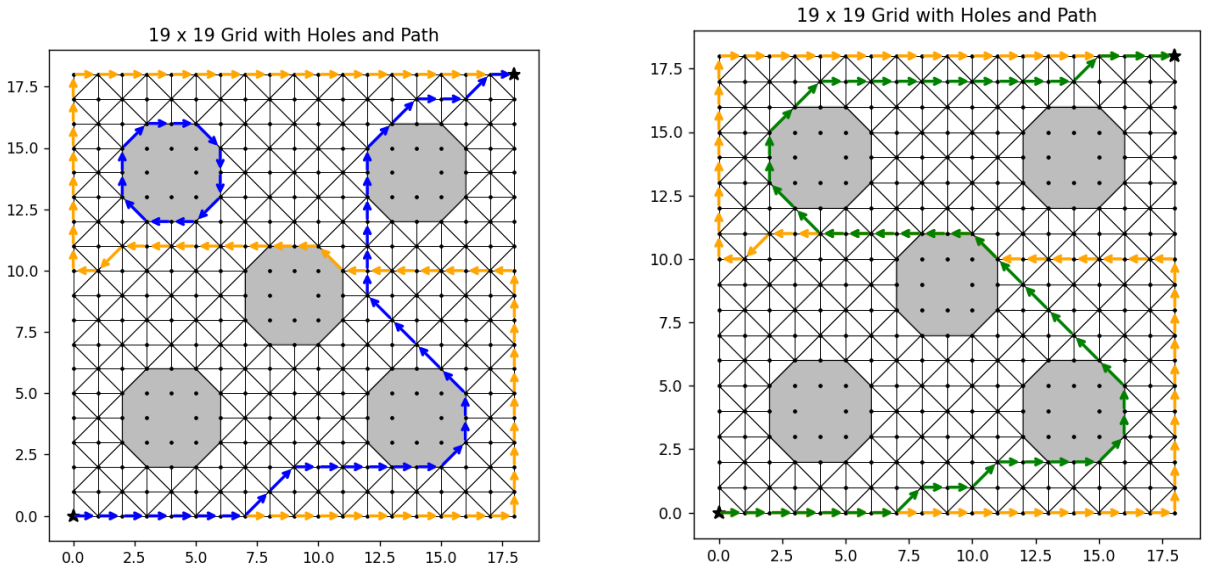
By Lemma 1, (1_{LP}) and (2_{LP}) have the same feasible region (up to the sign and box constraints on x).

Corollary 2. (2_{LP}) *has an integral optimal solution that is optimal for* (1).

Proof. By Lemma 1, (1) and (2) are equivalent, and (1) has the same optimal value as (1_{LP}). Since (1_{LP}) and (2_{LP}) describe the same feasible region, the claim follows. □

2.3 Paths with Cycles

We use the term *c-path* for a walk that contains cycles. A problem that arises in practice is that optimal solutions of (2_{LP}) may be c-paths rather than paths. Consider the example in Figure 1 with $s = (0,0)$ and $t = (18,18)$. The yellow path is the reference path P_{ref}, the blue path P_{true} is the optimal solution of (2_{LP}), and the green path $P_{expected}$ is the optimal simple path.



(a) Optimal c-path P_{true}

(b) Optimal simple path $P_{expected}$

Figure 1: The optimal solution of (2_{LP}) is not always a simple path.

Here P_{true} consists of a main (s,t)-dipath and one or more detached cycles, with $c(P_{true}) = 51.8$, whereas $c(P_{expected}) = 55.8$. Both are homologous to P_{ref}.

Proposition 3. *Let P_{true} consist of a main (s,t)-dipath and a detached cycle C around a hole H, and let $P_{expected}$ be a simple path homologous to P_{true} that leaves the main dipath at a node a, approaches H, partially loops around it, and rejoins the main dipath at a node b. Then the objective of (2_{LP}) prefers P_{true} to $P_{expected}$ if and only if forming the detached cycle is cheaper than the detour, i.e., if and only if inequality (1) below holds.*

Proof. Write $P_{true} = (s, v_1, \dots, a, \dots, b, \dots, v_n, t) + (v_{c,1}, \dots, v_{c,\ell}, v_{c,1})$, a main dipath plus a detached cycle, and write $P_{expected}$ as $s \to a \to v_{c,q} \to v_{c,r} \to b \to t$, where $v_{c,q} \to v_{c,r}$ follows part of C. The two paths share the segments $s \to a$ and $b \to t$, so $c(P_{true}) < c(P_{expected})$ is equivalent to

$$c(a \to b) + c(C) \;<\; c(a \to v_{c,q}) + c(v_{c,q} \to v_{c,r}) + c(v_{c,r} \to b). \tag{1}$$

□

In particular, whenever (1) holds for every simple alternative, every optimal solution of (2_{LP}) is a c-path.

This motivates the problem we study in the rest of the paper.

HOMOLOGOUS-SIMPLE-PATH (HSP). Given a reference path P_{ref}, find a minimum-cost *simple* path P^* homologous to P_{ref}.

2.4 Hardness

We believe HSP is NP-hard. We give a reduction from the (s,t)-Hamiltonian path problem; as explained in Remark 5, the gadget below is a sketch and does not yet constitute a complete proof.

Proposition 4 (NP-hardness of HSP, proof sketch)**.** *The decision version of* HSP *is NP-hard.*

Proof sketch. We reduce the (s,t)-Hamiltonian path problem on $G=(V,E)$ to the decision version of HSP, showing that an (s,t)-Hamiltonian path exists if and only if a homologous simple path of cost at most B exists, for a suitable budget B.

1. **Reduction.** Build a simplicial complex $\mathcal{X}=(\mathcal{V},\mathcal{E},\mathcal{T})$ with vertex set $\mathcal{V}=V\cup\{x_{1v},x_{2v},x_{3v}\}_{v\in V}$ and edge set $\mathcal{E}=E\cup\{(v,x_{1v}),(x_{1v},x_{2v}),(x_{2v},x_{3v}),(x_{3v},x_{1v}),(x_{1v},v)\}_{v\in V}$, with unit cost on every edge. The cycle (x_{1v},x_{2v},x_{3v}) is left unfilled, forming a hole. This creates $|V|$ holes, one adjacent to each original vertex, and the gadget at v is reachable only through v. The reference path is chosen so that the problem becomes finding a path τ with harmonic projection $H^\top\tau=[1,\dots,1]$, i.e., one that winds once around every hole.
2. **Hamiltonian path $\Rightarrow$ HSP.** A Hamiltonian path in G visits every $v\in V$ once; at each v it can wind around the gadget and return to v before continuing. The resulting path winds around every hole, so $H^\top\tau=[1,\dots,1]$.
3. **HSP $\Rightarrow$ Hamiltonian path.** A feasible path must wind around every hole, and to reach a hole it must reach the corresponding original vertex. Hence it visits every $v\in V$, and, being simple, it is a Hamiltonian path in G.

□

Remark 5. The gadget above is incomplete. Winding around the hole at v requires returning to x_{1v} and to v, which a *simple* path cannot do, so step 2 does not produce a simple path as written. A complete proof needs a gadget in which winding around a hole traverses fresh vertices, and must also account for the fact that $\mathcal{X}$ is planar (embedded in a disk), whereas G is arbitrary. We leave completing the reduction to future work, and regard NP-hardness of HSP as a well-motivated conjecture rather than a theorem.

3 A Heuristic Algorithm

Since HSP appears to be hard, we turn to a polynomial-time heuristic, ALG. Let $P(\tau,cost,G)$ denote the linear program that returns the cheapest unit flow from s to the terminal node of τ in the same homology class as τ, given the cost vector *cost* and graph G. For $\tau=\bar{\tau}$ this is the (s,t)-flow of (2_{LP}). ALG is defined in Algorithm 1.

Algorithm 1 ALG

Require: $G = (V, E)$, reference path $\bar{\tau}$
Ensure: A path τ^* homologous to $\bar{\tau}$
1: $cost \leftarrow \{(i,j) : c_{ij}\}_{(i,j)\in E}$, where c_{ij} is the cost of edge (i,j)
2: $\tau^* \leftarrow P(\bar{\tau}, cost, G)$
3: **if** τ^* contains a cycle **then**
4: $\quad i \leftarrow 1$
5: $\quad$ **while** $i < |\bar{\tau}|$ **do**
6: $\quad\quad \tau_{curr} \leftarrow \bar{\tau}[0 : i+1]$
7: $\quad\quad \tau^*_{curr} \leftarrow P(\tau_{curr}, cost, G)$
8: $\quad\quad$ **if** τ^*_{curr} contains a cycle **then**
9: $\quad\quad\quad \tau_{curr} \leftarrow \tau_{curr}[0 : -1]$
10: $\quad\quad\quad \tau^*_{curr} \leftarrow P(\tau_{curr}, cost, G)$
11: $\quad\quad\quad$ **for** $(j,k) \in \tau^*_{curr}$ **do**
12: $\quad\quad\quad\quad cost[(j,k)] \leftarrow 0$
13: $\quad\quad\quad\quad cost[(k,j)] \leftarrow 0$
14: $\quad\quad\quad$ **end for**
15: $\quad\quad$ **else**
16: $\quad\quad\quad i \leftarrow i+1$
17: $\quad\quad$ **end if**
18: $\quad$ **end while**
19: $\quad$ **return** τ^*_{curr}
20: **else**
21: $\quad$ **return** τ^*
22: **end if**

Lemma 6. *Each iteration of the while loop of ALG runs in polynomial time. Consequently, ALG runs in polynomial time whenever the zeroing step (lines 10–14) is triggered a polynomial number of times in total.*

Proof. The LP has $O(|E|)$ variables and $O(|V|) + D$ constraints, where D is the number of holes. Each call to $P(\tau, cost, G)$ solves a min-cost flow LP with homology constraints, which can be done in polynomial time [3]. Within an iteration we also search for a cycle in the output and update *cost*; both take polynomial time. If every index i triggers the zeroing step at most once, the loop executes $O(|\bar{\tau}|) = O(|V|)$ iterations. □

Remark 7. Lemma 6 is conditional because, after the zeroing step, the loop re-examines the same index i. If the zeroing does not remove the cycle, the state does not change and the loop repeats. We have not proven that this cannot happen, and in an implementation one should cap the number of iterations per index.

Figure 2 shows an output of ALG: the yellow path is the reference path $\bar{\tau}$ and the green path is the output τ^*_{curr}.

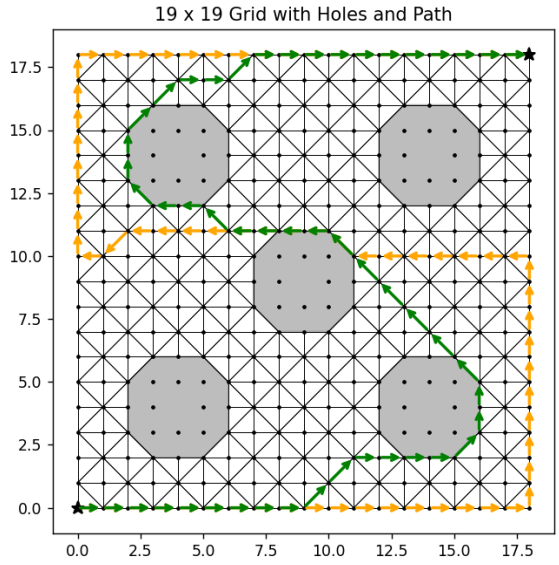


Figure 2: Output of ALG on an instance where (2_{LP}) returns a c-path.

3.1 What We Can and Cannot Prove

We would like ALG to return a path that is (1) in the same homology class as $\bar{\tau}$ and (2) simple. We can prove the first property.

Theorem 8. *If ALG terminates, its output lies in the same homology class as the reference path $\bar{\tau}$.*

Proof. The call $P(\bar{\tau}, cost, G)$ in line 2 returns a path τ^* in the same homology class as $\bar{\tau}$, by the homology constraint in P. If it contains no cycle, ALG returns τ^*.

Otherwise, the while loop only changes the cost vector and leaves G unchanged, so the feasible region of P is unchanged. Moreover, the loop exits only after an iteration with $i = |\bar{\tau}| - 1$ in which $\tau_{curr} = \bar{\tau}$ and the call $P(\bar{\tau}, cost, G)$ returns a cycle-free path. By the homology constraint, this path lies in the same homology class as $\bar{\tau}$. □

For intuition about why the prefix-by-prefix construction is natural, we record the following fact. Let τ_{i-1} be an (s, t_{i-1})-path, let $\tau_i = \tau_{i-1} \cup \{(t_{i-1}, t_i)\}$, and let τ^*_{i-1} be another path in the same homology class as τ_{i-1}.

Lemma 9. *$\tau^*_{i-1} \cup \{(t_{i-1}, t_i)\}$ lies in the same homology class as τ_i.*

Proof. If $\tau^*_{i-1} = \tau_{i-1}$ there is nothing to show, so assume otherwise. Then $H^\top \tau^*_{i-1} = H^\top \tau_{i-1}$, and

$$H^\top \tau_i = H^\top \tau_{i-1} + H^\top \{(t_{i-1}, t_i)\} = H^\top \tau^*_{i-1} + H^\top \{(t_{i-1}, t_i)\} = H^\top \big(\tau^*_{i-1} \cup \{(t_{i-1}, t_i)\}\big). \qquad \square$$

Simplicity is not proven. We do not have a proof that ALG always returns a simple path. The natural argument is that zeroing the cost of the prefix τ^*_{i-1} forces every optimal solution of the recomputed LP to contain that prefix, and that any additional cycle then strictly increases the cost. Both steps fail in general:

1. Zero cost on the prefix does not force it to be used: a route avoiding the prefix may still be cheaper than the prefix followed by a long completion to the terminal node.
2. Zeroing both (j, k) and (k, j) creates zero-cost two-cycles, so adding a cycle only *weakly* increases the objective and cycles can be optimal.

On the instances we examined, such as Figure 2, ALG returned a simple path, but we treat this as an empirical observation. Whether a variant of ALG with a provable simplicity guarantee exists is open.

4 Suboptimality of ALG

ALG is a heuristic and can be far from optimal. In Figure 3, the reference path is in yellow, the output of ALG is in green, and the true optimal simple path is in blue.

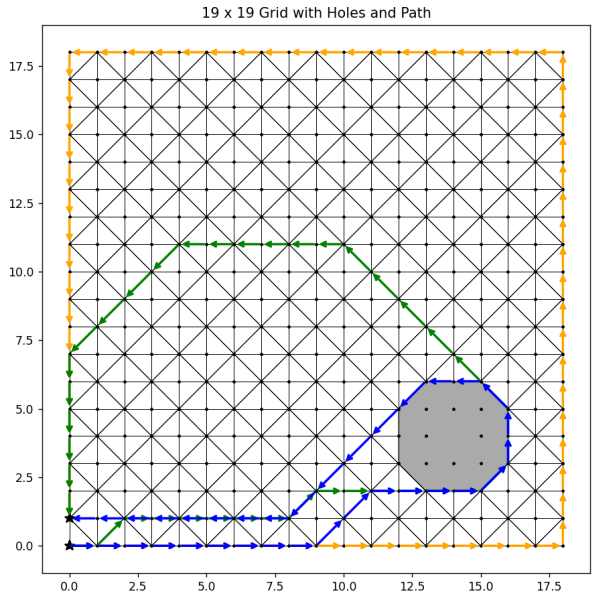


Figure 3: An instance on which ALG's output is suboptimal.

In fact, there is no constant-factor guarantee.

Theorem 10. *ALG has an unbounded approximation ratio for* HSP*: for every $\rho > 0$ there is an instance on which cost*(ALG)/*cost*(OPT) $> \rho$.

Proof. Consider the directed graph in Figure 4, with $s = v_1$ and $t = v_2$, where the yellow edges form the reference path and $M > 0$ is a parameter. The optimal path is $\{(v_1, v_4), (v_4, v_7), (v_7, v_3), (v_3, v_2)\}$

with cost 4. ALG outputs $\{(v_1, v_4), (v_4, v_7), (v_7, v_5), (v_5, v_6), (v_6, v_1)\}$ with cost $M + 4$. Therefore

$$\frac{cost(\text{ALG})}{cost(\text{OPT})} = \frac{M+4}{4} \longrightarrow \infty \quad \text{as } M \to \infty. \qquad \square$$

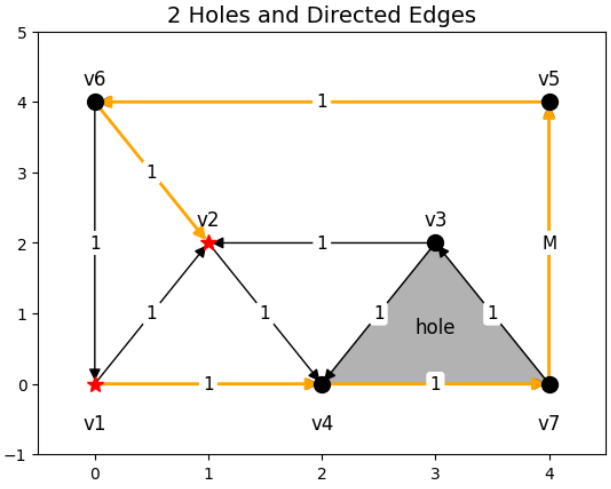


Figure 4: Instance on which ALG's cost is arbitrarily larger than optimal. Yellow edges form the reference path.

5 Exact Formulations of HSP

Since HSP appears to be hard, exact methods are necessarily exponential in the worst case. We describe two such formulations that we used in our experiments.

5.1 MTZ Constraints

The Miller–Tucker–Zemlin (MTZ) constraints are linear constraints that eliminate subtours in the traveling salesman problem [4]. We use them to prevent detached cycles in P_{true}. Introducing

ordering variables π_v, we augment (2) as

$$\min \quad \sum_{(a,b)\in\mathcal{E}} c_{ab}x_{ab} \tag{2_{MTZ}}$$

$$\text{s.t.} \quad \sum_{e\in\delta^-(u)} x_e - \sum_{e\in\delta^+(u)} x_e = \begin{cases} -1 & u = s, \\ 1 & u = t, \\ 0 & \text{otherwise} \end{cases}$$

$$H^\top \tau(x) = H^\top \bar{\tau}$$

$$\pi_s = 0 \tag{MTZ I}$$

$$\pi_a - \pi_b + |\mathcal{V}|\, x_{ab} \le |\mathcal{V}| - 1 \quad (\forall (a,b) \in \mathcal{E}) \tag{MTZ II}$$

$$0 \le \pi_v \le |\mathcal{V}| - 1 \quad (\forall v \in \mathcal{V}) \tag{MTZ bounds}$$

$$x_{ab} \in \{0,1\} \quad (\forall (a,b) \in \mathcal{E}).$$

The MTZ constraints enforce an ordering, starting from s, on the nodes used by the path. Nodes on a cycle cannot be consistently ordered, so detached cycles are infeasible; summing (MTZ II) around a cycle C gives $|\mathcal{V}||C| \le (|\mathcal{V}| - 1)|C|$, a contradiction. Hence every feasible solution is a simple path homologous to $\bar{\tau}$, and (2) with MTZ constraints solves HSP exactly. The drawback is that this is a mixed-integer program, since the constraint matrix is no longer totally unimodular.

5.2 Cycle Cuts

Alternatively, we can solve (2_{LP}), check the solution for a cycle C, add a linear constraint that forbids C, and repeat. The resulting program after one round of cuts is

$$\min \quad \sum_{(a,b)\in\mathcal{E}} c_{ab}x_{ab} \tag{2_{cuts}}$$

$$\text{s.t. flow conservation and homology constraints as in } (2_{LP}),$$

$$\sum_{(a,b)\in C} x_{ab} \le |C| - 1 \quad \text{for each cycle } C \text{ found so far} \tag{cutting plane}$$

$$x_{ab} \in \{0,1\} \quad (\forall (a,b) \in \mathcal{E}).$$

Each cut forbids only a single cycle, so many iterations may be needed before all cheap cycles around a hole H have been excluded and the model settles on a simple path.

6 Open Problems

We leave the following questions open.

1. **Hardness.** Complete the NP-hardness proof for HSP (Remark 5): construct a gadget

through which a simple path can wind around a hole, and handle the planarity of the underlying complex.

2. **Fixed number of holes.** Is HSP polynomial-time solvable when the number of holes D is constant? Is it fixed-parameter tractable in D?

3. **Heuristics with guarantees.** Does any polynomial-time algorithm achieve a bounded approximation ratio for HSP, or is the problem hard to approximate? Theorem 10 rules this out only for ALG.

4. **Correct variants of ALG.** Is there a modification of ALG (for instance, one that does not zero reverse edges) for which termination and simplicity can be proven?

5. **Exact methods in practice.** How do MTZ formulations and lazily separated cycle cuts compare to the Lagrangian approach of [1] in running time and solution quality?

References

[1] C. A. Taveras, S. Segarra, and C. A. Uribe. "Efficient Path Planning with Soft Homology Constraints". In: *Proceedings of the IEEE Conference on Decision and Control (CDC)*. 2022, pp. 1–6. DOI: 10.1109/CDC51059.2022.9992793.

[2] T. K. Dey, A. N. Hirani, and B. Krishnamoorthy. "Optimal Homologous Cycles, Total Unimodularity, and Linear Programming". In: *SIAM Journal on Computing* 40.4 (2011), pp. 1026–1044. DOI: 10.1137/09076637X.

[3] Narendra Karmarkar. "A new polynomial-time algorithm for linear programming". In: *Combinatorica* 4.4 (1984), pp. 373–395.

[4] C. E. Miller, A. W. Tucker, and R. A. Zemlin. "Integer Programming Formulation of Traveling Salesman Problems". In: *Journal of the ACM* 7.4 (1960), pp. 326–329.